Retrieval-Augmented Generation (RAG)

Agentic AI, February 23rd

RECAP : Memory Management

Memory management in AI agents bridges the gap between stateless LLMs and persistent, context-aware systems by combining short-term memory (real-time but token-limited) with long-term memory (vector DBs and knowledge graphs).

Hybrid approaches use intelligent routing to balance both, while user-level memory enables personalization but requires careful privacy safeguards.

User memory and personalization lets agents adapt tone, recommendations, and behavior based on individual user history. But this creates tension with privacy — systems need clear data retention policies, user control over what's stored, and safeguards against leaking personal data across sessions or users.

Content for today

- RAG Architectures
- Chunking strategies
- Embeddings tradeoffs
- GraphRAG
- Failure Modes of RAG

Why RAG?

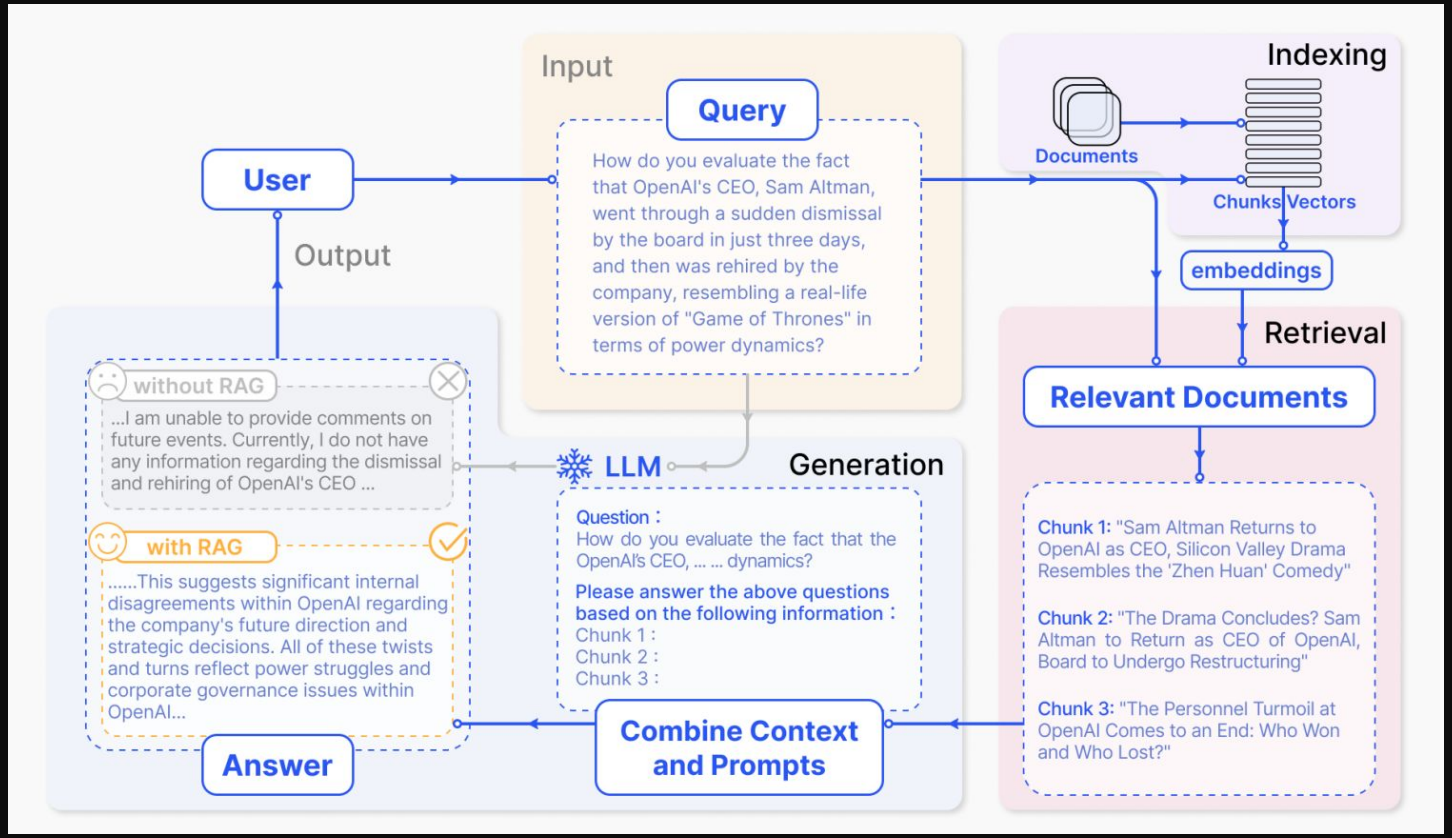
- Reducing hallucination
- Help with outdated Information used in training
- Help explain non-transparent, untraceable reasoning processes

What is RAG?

Retrieving information from an external database and feed to a model to enhance its performance

This is most commonly done at inference time

Typical RAG Application



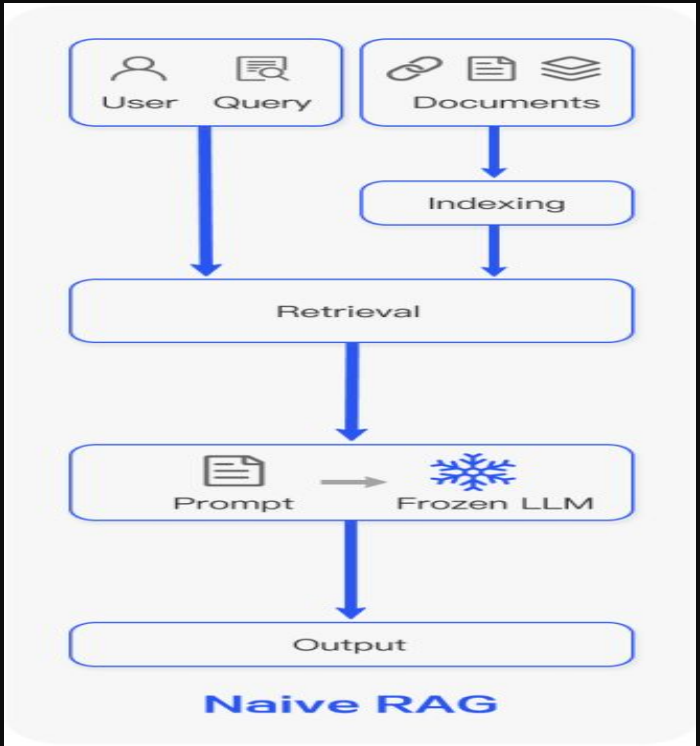
Development Timeline of RAG

Naive RAG	Advanced RAG	Modular RAG	Agentic RAG and more
Traditional process of: - Indexing - Retrieval - Generation “Retrieve-Read” framework	Enhance specific parts of naive RAG, adding - Pre-retrieval - Post-retrieval	Enhanced adaptability/versatility - Modularization of different stages - New enhancement modules - New patterns	Newer RAG tools incorporate the agentic structure, allowing models to decide when to invoke RAG processes

RAG Architectures

Naive RAG

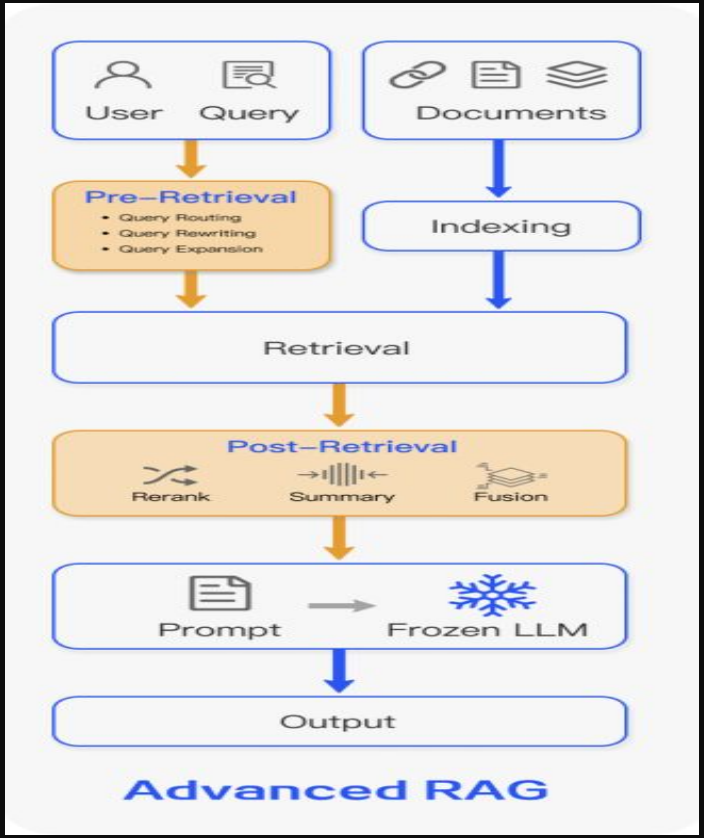
- Indexing
  - Cleaning/extracting plain text from different formats
  - Chunking → embedding model → vector representation
- Retrieval
  - Query → embedding model → vector representation
  - Similarity score with chunks → return top K chunks
- Generation
  - Add both retrieved info and query to a prompt
  - Input to model for response



Advanced RAG

Additional Stages

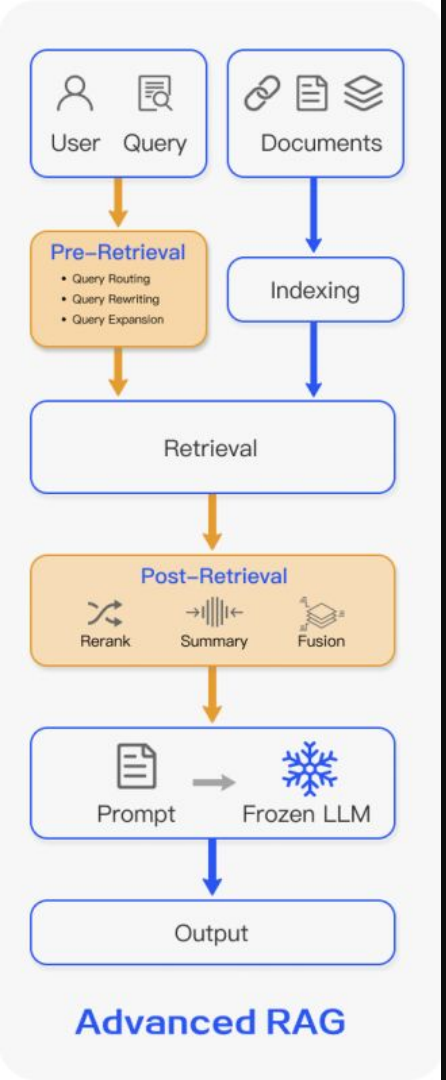
- Pre-Retrieval
  - Optimizing Indexing:
    - Enhancing Data Granularity: chunk size/segmentation
    - Optimizing Index Structures: embedding storage/search
    - Adding Metadata: tags, timestamps, etc.
    - Alignment Optimization: align embedding space with generation objectives. E.g. Instruction-tuned embedding models
    - Mixed Retrieval: mix of dense/sparse search
  - Query Optimization: (often LLM assisted)
    - Query Rewriting: removing conversational noise
    - Query Transformation: reformatting query
    - Query Expansion: add synonyms and other semantic hints



Advanced RAG (Cont.)

Additional Stages

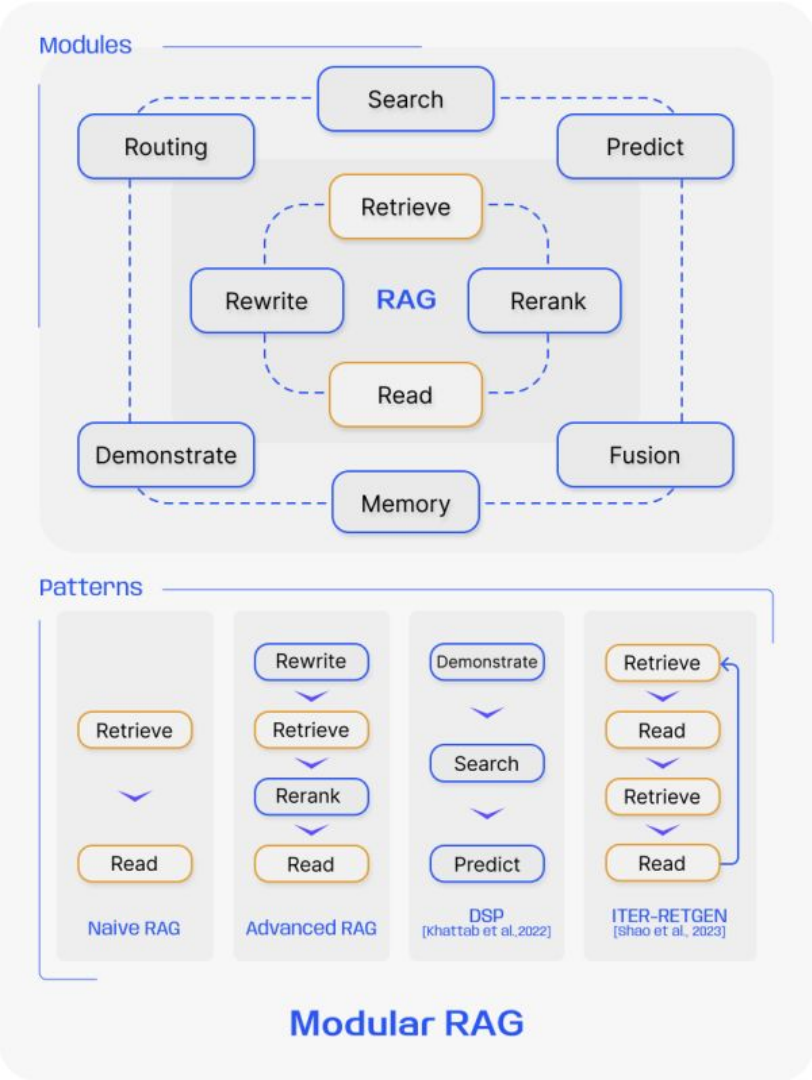
- Post-Retrieval
  - Re-rank chunks:
    - Cross-Encoder: encodes (query,chunk), output relevance score
    - LLM-based, Heuristic based (e.g. prefer recent)
    - Ranking on relevance to edges of prompt
  - Context Compressing:
    - Extractive Compression: keyword overlap
    - Abstractive Compression: LLM summarization



Modular RAG

New Modules:

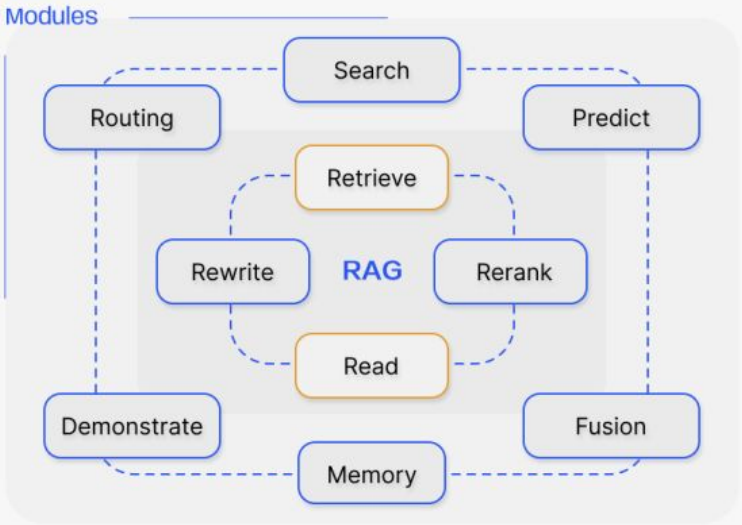
- Search: adapts to search in different data sources (e.g. docs, databases, knowledge graphs)
- Fusion: Generate multiple queries, run independently, merge results
- Memory: Stores interaction history, retrieved documents, etc. Feed back into system when needed
- Routing: Select the optimal retrieval or processing pathway, e.g. what search/query type
- Predict: Reduce redundancy - LLM first generate target intermediate context for query, then used to search
- Task Adapter: Fine-tunes / configures retriever for specific task

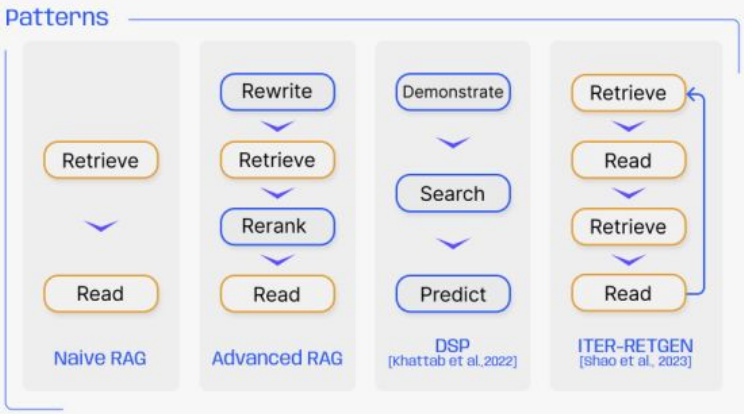


Modular RAG (Cont.)

New Patterns:

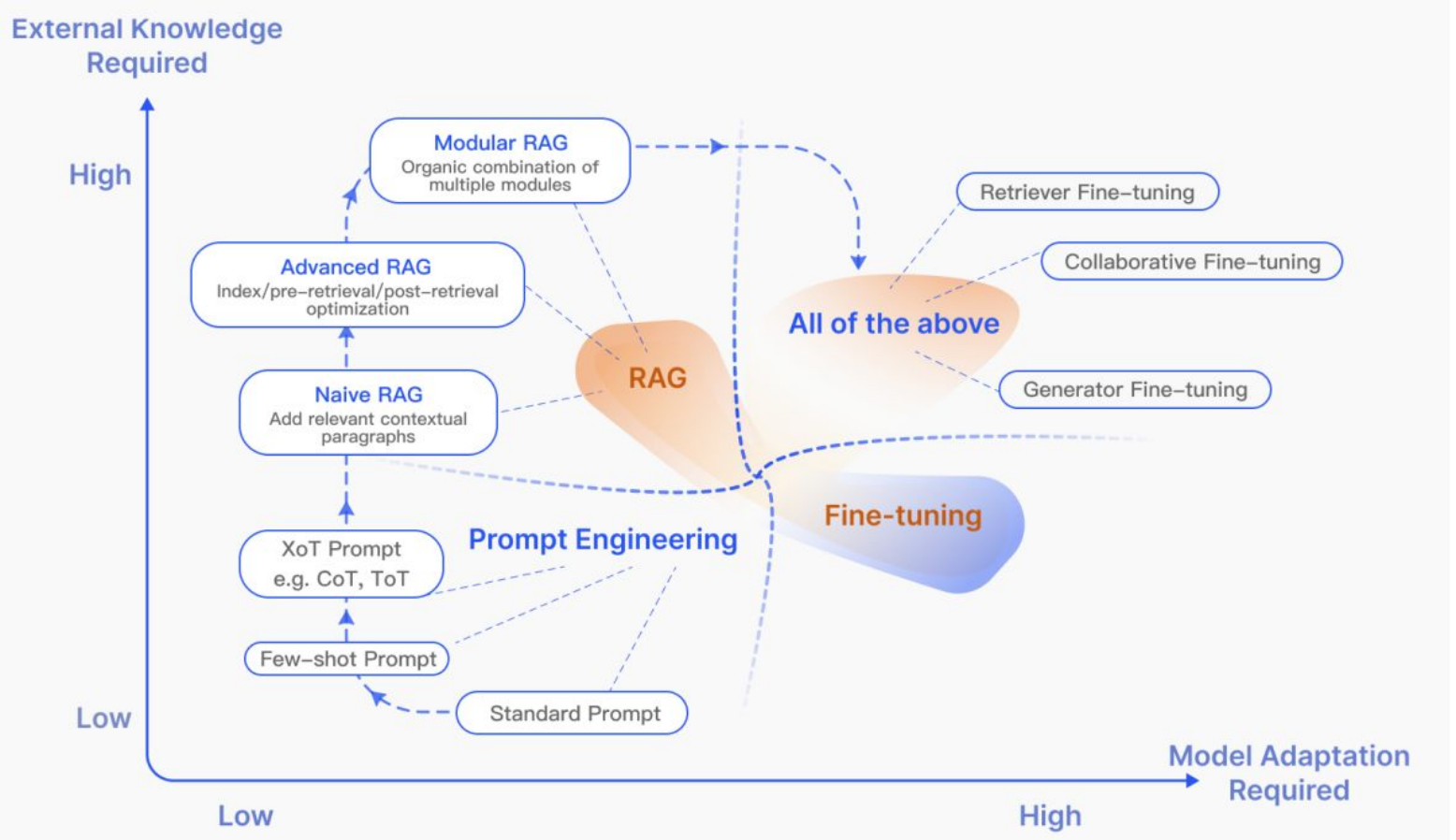
- Rewrite–Retrieve–Read: add rewrite model before retrieval, feedback updates rewrite module
- Generate–Read/Recite–Read: remove external data, generate relevant context (better knowledge recall)
- Hybrid Retrieval: keyword, semantic, and vector searches
- DSP (Demonstrate–Search–Predict): demonstrations or examples guide search. Search results inform prediction.
- ITER-RETGEN (Retrieve–Read–Retrieve–Read): Retrieval and generation alternate iteratively
- Many more!





Modular RAG

RAG vs Prompt Engineering vs Fine Tuning



Pros and Cons

RAG: Pros

Easy knowledge updates, medium cost, good for knowledge-intensive tasks, interpretable, handles unseen info

RAG: Cons

Higher latency, infrastructure complexity, retrieval noise

Prompt Engineering: Pros

Low cost, low latency, no infrastructure, immediate, works well for reasoning

Prompt Engineering: Cons

Limited to model's knowledge, weak control over deeper behavior alignment

Fine-Tuning: Pros

Deep behavioral control, medium latency, can reduce hallucination in narrow domains

Fine-Tuning: Cons

Expensive, static knowledge, risk of overfitting, less interpretable

Topics Deepdive

Chunking Strategies

Chunking Strategies

Many different strategies, we'll only cover the most common ones

- Fixed-size chunking
- Simple Content-Aware Splitting
- Recursive Character Level Chunking
- Document structure-based chunking
- Semantic Chunking
- Contextual Chunking with LLMs

Fixed-Size Chunking (A)

- Split documents into equal token windows
- If overlap is included, sometimes called “sliding window”
- Pros:
  - Strong default for most RAG systems
  - Fast preprocessing
  - Easy to debug and reason about
- Cons:
  - Ignores document structure
  - May split mid-sentence or mid-section
  - Can mix unrelated ideas inside one chunk
- When to Use: Start here. Only move beyond this if retrieval quality is insufficient.

Simple Content-Aware Splitting (B)

- Break text using linguistic boundaries:
  - Periods or newline splitting (naive)
  - Or more complex like NLTK sentence tokenizer
- Pros:
  - Cleaner semantic units
  - Works well for sentence-optimized embedding models
  - Low complexity increase
- Cons:
  - Sentence-level chunks can be too small
  - Paragraphs may still be too coarse
  - No guarantee of balanced token sizes
- When to Use: FAQs, Blog-style text, Sentence-optimized embedding models

Recursive Chunking (A+B)

- Split using ordered separators (e.g., paragraph → newline → space → fallback), while enforcing max chunk size.
- Pros:
  - Better coherence than fixed-only
  - More consistent size than sentence-only
  - Strong practical default
- Cons:
  - Still surface-level (not semantic-aware)
  - Requires tuning chunk size and overlap
- When to Use: Excellent production default when fixed chunking feels too naive. (Some benchmarks show this performs best among many methods)

Other Chunking Methods

- Document Structure-Based Chunking: chunking per page/section/tags/headings. Depend on original file format. Coherent but uneven chunk size
- Semantic Chunking: split into sentence, embed, compare topic similarity. Coherent chunks, but compute heavy, inconsistent size, hard to tune
- Contextual Chunking with LLMs: Basically what is sounds like. Good performance but highest cost of all

Embeddings

Embedding Tradeoffs

Embeddings determine: How meaning is represented in vector space and what “similarity” means

Model Size

- Small: Cheap and fast, more noise, less semantic resolution
- Large: slower and more costly, but better semantic understanding and domain generalization

Embedding Dimensionality

- High: Finer semantic separation, higher cost, slower search, risk of overfitting
- Low: Faster, cheaper, lower semantic fidelity

Embedding Tradeoffs (Cont.)

Domain-Specific vs General

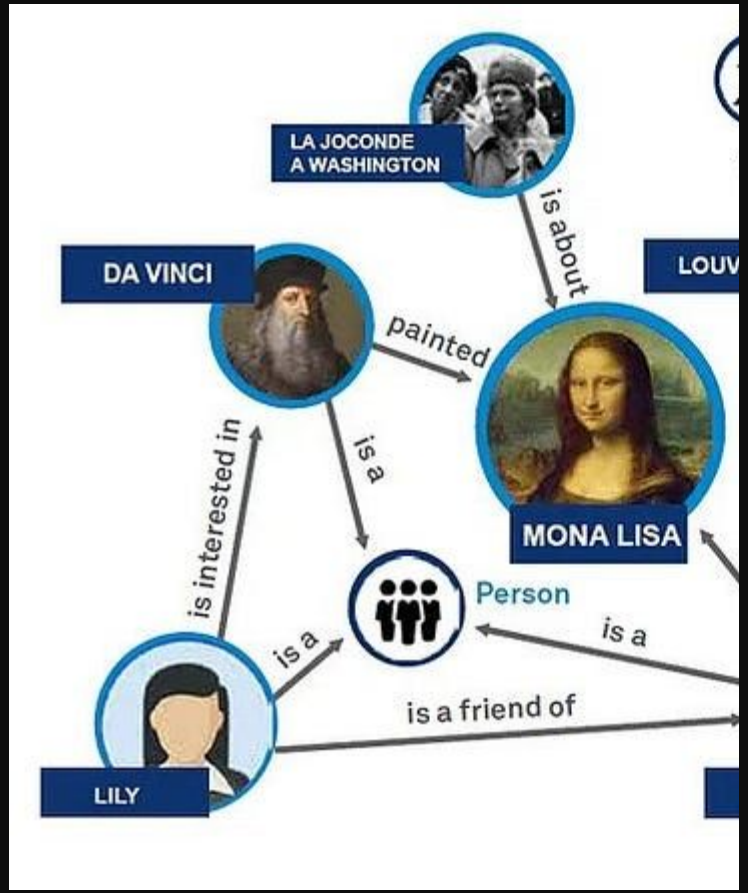
- General: Broad coverage, good zero-shot behavior, may collapse technical distinctions
- Specific: Better separation for specialized terminology, expensive to train, poor generalization outside domain

Similarity Metric

- Most vector DBs use: Cosine similarity, Dot product, Euclidean distance
- In practice: Cosine similarity is most stable, Dot product works well with normalized vectors
- Choice affects: Score distribution, Ranking behavior

Graph RAG

- augments retrieval by using knowledge graphs (entities + relationships)
- Query → entities → traverse relationships → subgraph
- Components:
  - Entity extraction (people, concepts, events)
  - Relationship extraction (who did what to whom)
  - Knowledge graph construction
  - Graph traversal at query time
  - LLM synthesis over retrieved subgraph
- Pros:
  - Strong for complex relational queries
  - Good for enterprise / structured knowledge
  - More controllable reasoning flow
- Cons
  - Heavy preprocessing cost
  - Graph construction is error-prone
  - Higher system complexity
  - Harder to scale and maintain



Failure modes of RAG

Failure modes of RAG

- Retrieval Failure (Garbage In): irrelevant chunks returned
  - Causes: Poor embeddings, bad chunking strategy, low-quality indexing, query–document vocabulary mismatch
  - Result: Model hallucinates or answers vaguely
- Context Fragmentation: relevant info split across chunks, usually due to bad chunking, result in incomplete answers/reasoning
- Over-Retrieval / Noise Injection: Too many weakly relevant chunks are retrieved
  - Causes: Large top-k, low similarity thresholds, high-dimensional embedding noise
  - Result: Model becomes distracted, inconsistent, or contradictory

Failure modes of RAG (Cont.)

- Stale or Incorrect Source Data
- Prompt–Retrieval Misalignment: Retrieved evidence exists but model ignores or misuses it
  - Cause: Weak instruction prompting, poor context formatting, no citation constraints
  - Result: Model drifts beyond retrieved evidence

Project

Super Tiny RAG Agent

As simple as a RAG Agent can be:

- Loads in plain text
- Recursive chunking (fixed size created a bad chunk for our example)
- Embed with SentenceTransformer (all-MiniLM-L6-v2 model)
- Index and store with FAISS (lightweight, simple)
- LLM with Ollama (gemma3:4b)

Attendance

References

[1] https://doi.org/10.48550/arXiv.2312.10997
[2] https://www.pinecone.io/learn/chunking-strategies/#Chunking-methods